

DSNHackathon

September 27, 2026

```
[68]: #Data
import pandas as pd
import numpy as np

#Visualization
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_validate
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.ensemble import RandomForestRegressor
from sklearn.ensemble import GradientBoostingRegressor
from xgboost import XGBRegressor
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import uniform, randint
```

```
[85]: train = pd.read_csv(r"C:\Users\USER\Documents\DSN Hakathon\train.csv")
test = pd.read_csv(r"C:\Users\USER\Documents\DSN Hakathon\test.csv")
```

```
[4]: train.head()
```

```
[4]:
```

	id	product_code	product_weight_kg	fat_content	shelf_visibility	\
0	row_00000	PRD-PRFP9S	14.252	Low Fat	0.0271	
1	row_00001	PRD-PXXK71	7.698	Low Fat	0.0720	
2	row_00002	PRD-V5MOIJ	14.264	Regular	0.0421	
3	row_00003	PRD-UN5Z3J	12.638	Regular	0.0449	
4	row_00004	PRD-6RDQYB	10.338	Regular	0.0120	

	Proper Product_Category	product_price	store_code	store_age_years	\
0	Frozen Foods	81.37	STORE-AGY	45	
1	Health And Hygiene	42.05	STORE-YLW	35	
2	Canned	41.35	STORE-89Z	33	
3	Soft Drinks	174.35	STORE-7WS	47	

4 Meat 203.06 STORE-9RG 28

	store_size	store_location_tier		store_format	total_sales
0	Large	Tier_3	Standard	Supermarket	1764.98
1	Small	Tier_1	Standard	Supermarket	342.13
2	Medium	Tier_1	Standard	Supermarket	378.85
3	Medium	Tier_3	Flagship	Hypermarket	5595.72
4	Small	Tier_2	Standard	Supermarket	2375.36

```
[5]: train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6818 entries, 0 to 6817
Data columns (total 13 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   id                                    6818 non-null   object
1   product_code                         6818 non-null   object
2   product_weight_kg                    6818 non-null   float64
3   fat_content                          6818 non-null   object
4   shelf_visibility                     6818 non-null   float64
5   Proper Product_Category              6818 non-null   object
6   product_price                        6818 non-null   float64
7   store_code                           6818 non-null   object
8   store_age_years                      6818 non-null   int64
9   store_size                           6818 non-null   object
10  store_location_tier                  6818 non-null   object
11  store_format                         6818 non-null   object
12  total_sales                          6818 non-null   float64
dtypes: float64(4), int64(1), object(8)
memory usage: 692.6+ KB
```

```
[6]: train.shape
```

```
[6]: (6818, 13)
```

```
[7]: train.isnull().sum()
```

```
[7]: id                0
product_code         0
product_weight_kg     0
fat_content           0
shelf_visibility      0
Proper Product_Category 0
product_price         0
store_code            0
store_age_years       0
store_size            0
```

```
store_location_tier      0
store_format              0
total_sales              0
dtype: int64
```

```
[8]: train.duplicated().sum()
```

```
[8]: np.int64(0)
```

```
[9]: train['Proper Product_Category'].value_counts()
```

```
[9]: Proper Product_Category
Fruits And Vegetables    1006
Snack Foods              933
Household                740
Frozen Foods             675
Dairy                   554
Canned                  517
Baking Goods            516
Health And Hygiene      412
Soft Drinks             366
Meat                    339
Breads                  209
Hard Drinks             169
Others                  132
Starchy Foods          113
Breakfast               85
Seafood                 52
Name: count, dtype: int64
```

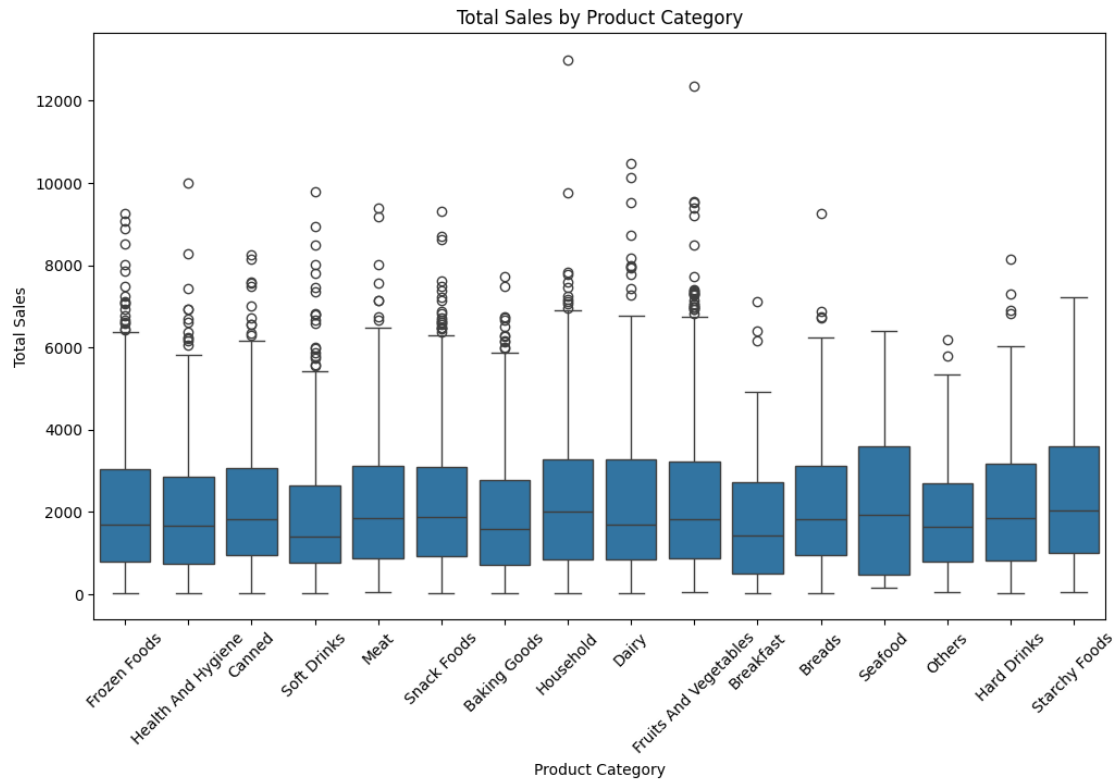
```
[10]: plt.figure(figsize=(12, 7))

sns.boxplot(
    data=train,
    x='Proper Product_Category',
    y='total_sales'
)

plt.title('Total Sales by Product Category')
plt.xlabel('Product Category')
plt.ylabel('Total Sales')

plt.xticks(rotation=45)

plt.show()
```



```
[11]: train['fat_content'].value_counts()
```

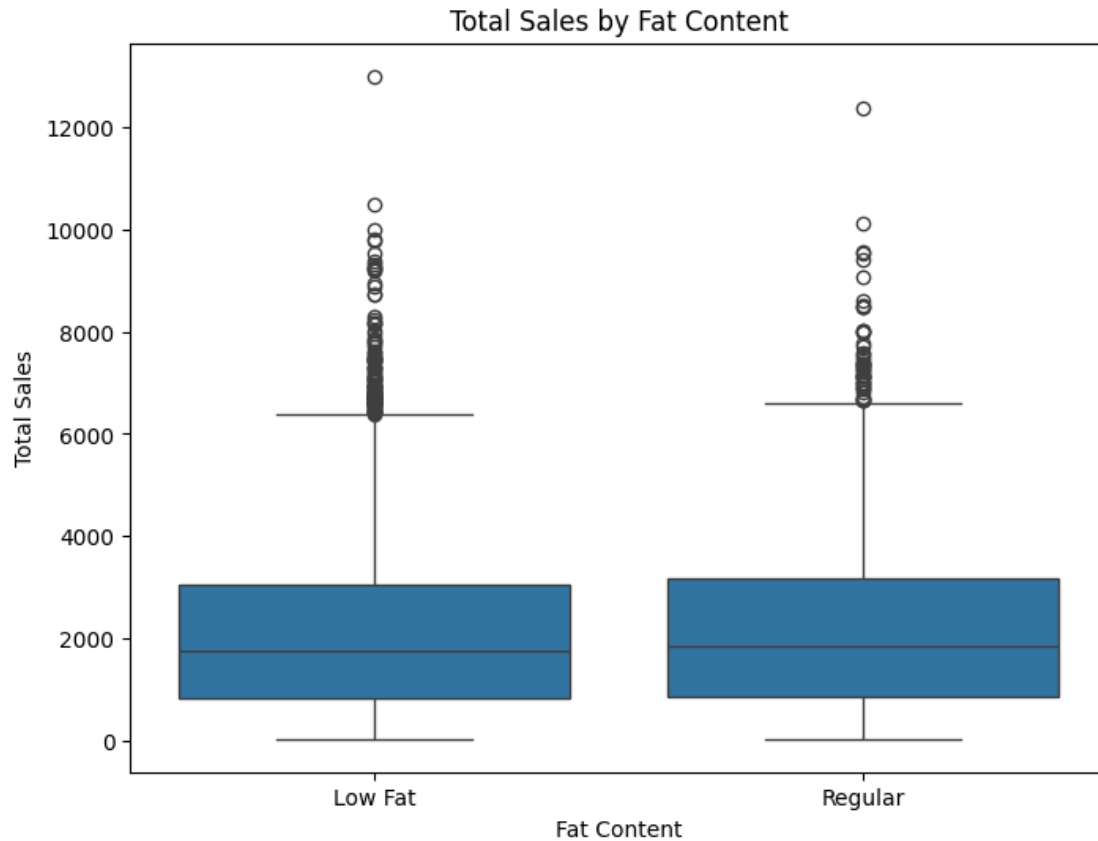
```
[11]: fat_content
Low Fat    4425
Regular    2393
Name: count, dtype: int64
```

```
[12]: plt.figure(figsize=(8, 6))

sns.boxplot(
    data=train,
    x='fat_content',
    y='total_sales'
)

plt.title('Total Sales by Fat Content')
plt.xlabel('Fat Content')
plt.ylabel('Total Sales')

plt.show()
```



```
[13]: train['store_size'].value_counts()
```

```
[13]: store_size
Medium    2218
Small     1933
Unknown   1919
Large      748
Name: count, dtype: int64
```

```
[14]: avg_sales_size = (
    train.groupby('store_size')['total_sales']
        .mean()
        .sort_values(ascending=False)
)

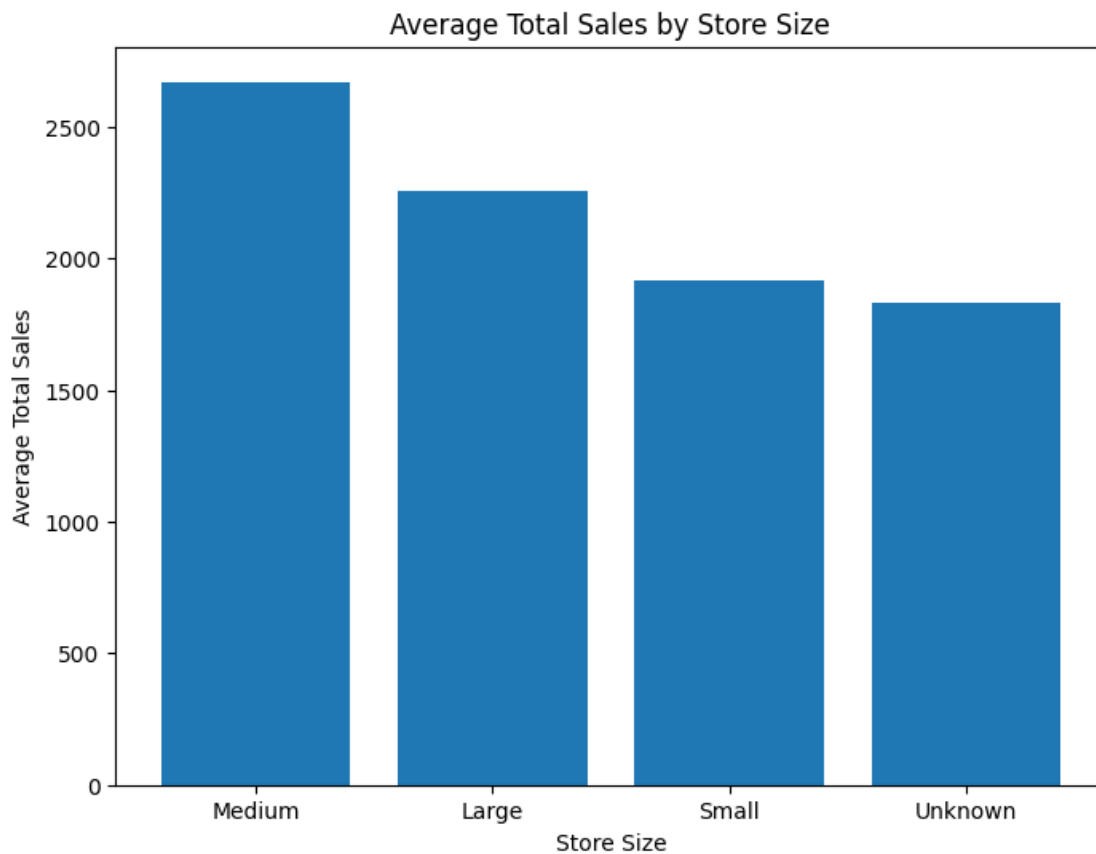
plt.figure(figsize=(8, 6))

plt.bar(
    avg_sales_size.index,
    avg_sales_size.values
```

```
)

plt.title('Average Total Sales by Store Size')
plt.xlabel('Store Size')
plt.ylabel('Average Total Sales')

plt.show()
```



```
[15]: train['store_location_tier'].value_counts()
```

```
[15]: store_location_tier
Tier_3    2677
Tier_2    2232
Tier_1    1909
Name: count, dtype: int64
```

```
[16]: plt.figure(figsize=(8, 6))
```

```
sns.boxplot(
    data=train,
```

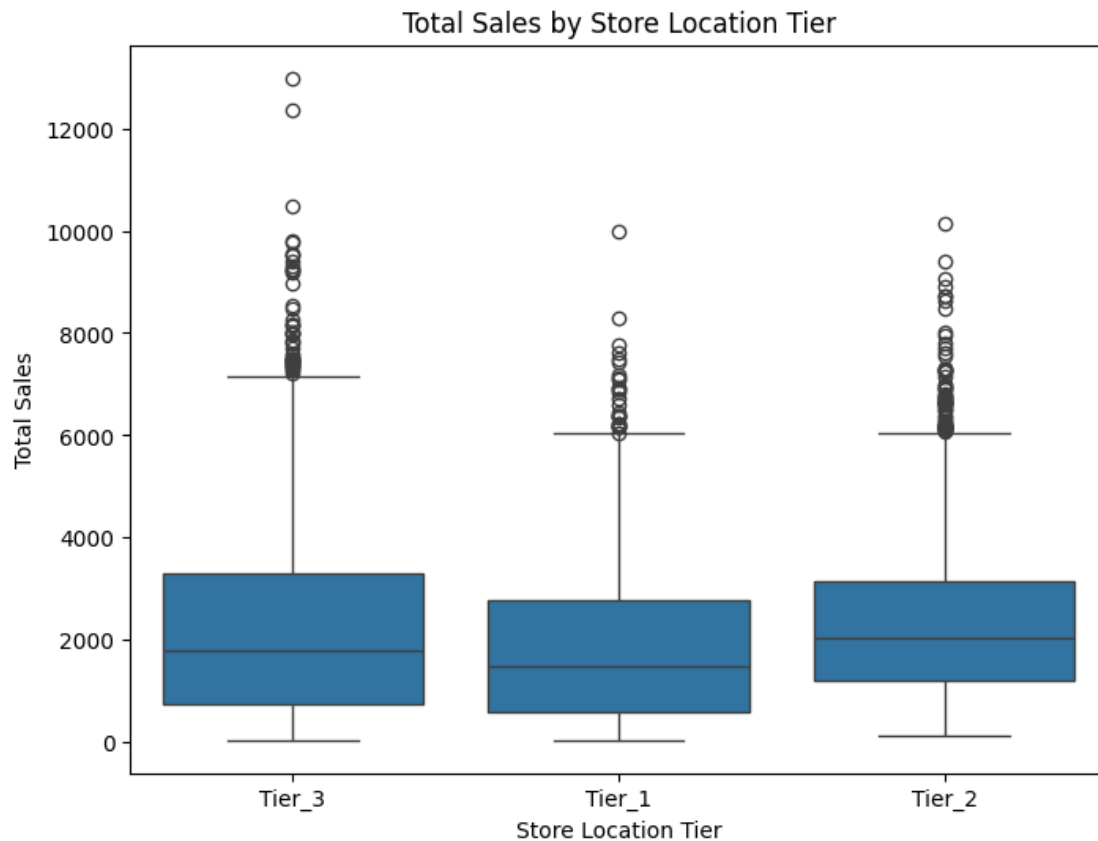
```

    x='store_location_tier',
    y='total_sales'
)

plt.title('Total Sales by Store Location Tier')
plt.xlabel('Store Location Tier')
plt.ylabel('Total Sales')

plt.show()

```



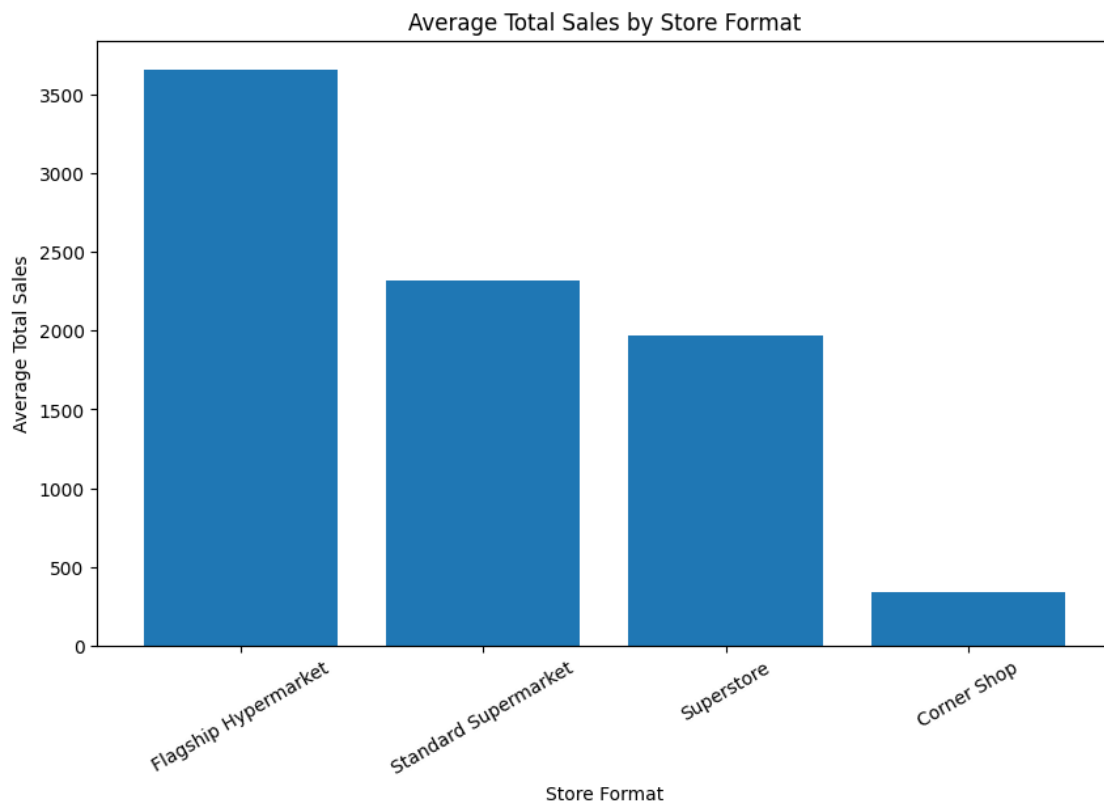
```
[17]: train['store_format'].value_counts()
```

```

[17]: store_format
Standard Supermarket    4462
Corner Shop              866
Flagship Hypermarket    748
Superstore              742
Name: count, dtype: int64

```

```
[18]: avg_sales_format = (  
        train.groupby('store_format')['total_sales']  
        .mean()  
        .sort_values(ascending=False)  
    )  
  
    plt.figure(figsize=(10, 6))  
  
    plt.bar(  
        avg_sales_format.index,  
        avg_sales_format.values  
    )  
  
    plt.title('Average Total Sales by Store Format')  
    plt.xlabel('Store Format')  
    plt.ylabel('Average Total Sales')  
  
    plt.xticks(rotation=30)  
  
    plt.show()
```



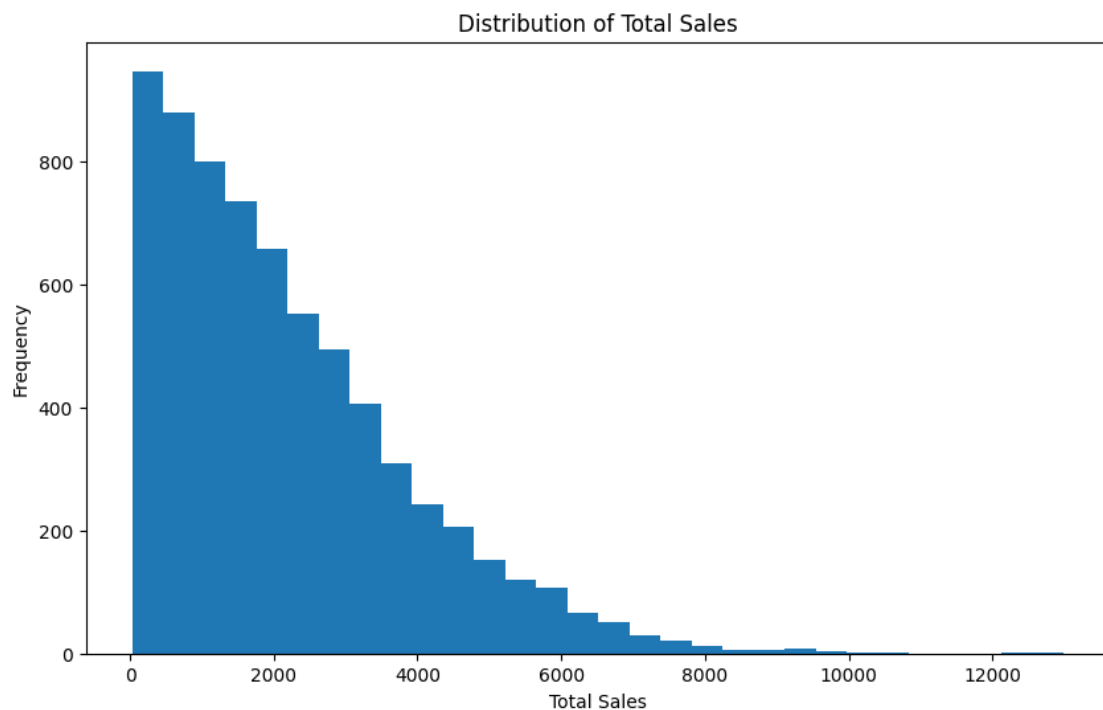

```
[19]: train['total_sales'].describe()
```

```
[19]: count      6818.000000  
      mean       2174.756574  
      std       1697.894956  
      min        32.700000  
      25%       834.792500  
      50%      1790.890000  
      75%      3092.587500  
      max      12996.820000  
      Name: total_sales, dtype: float64
```

```
[20]: train['total_sales'].skew()
```

```
[20]: np.float64(1.1539947519138116)
```

```
[21]: plt.figure(figsize=(10, 6))  
  
      plt.hist(train['total_sales'], bins=30)  
  
      plt.title('Distribution of Total Sales')  
      plt.xlabel('Total Sales')  
      plt.ylabel('Frequency')  
  
      plt.show()
```

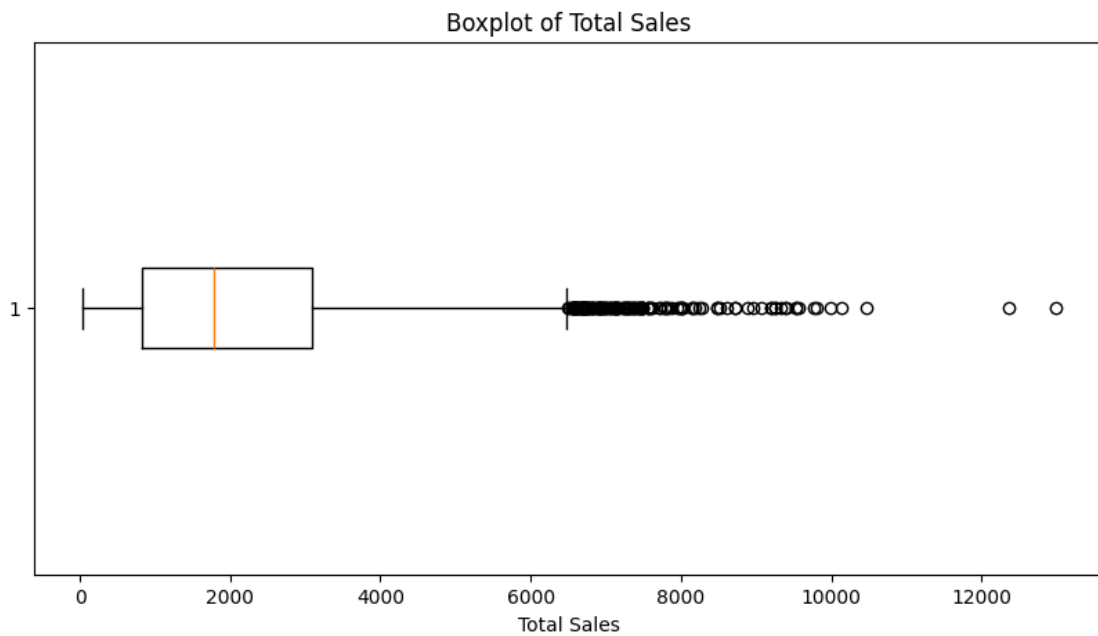


```
[22]: plt.figure(figsize=(10, 5))

plt.boxplot(train['total_sales'], vert=False)

plt.title('Boxplot of Total Sales')
plt.xlabel('Total Sales')

plt.show()
```

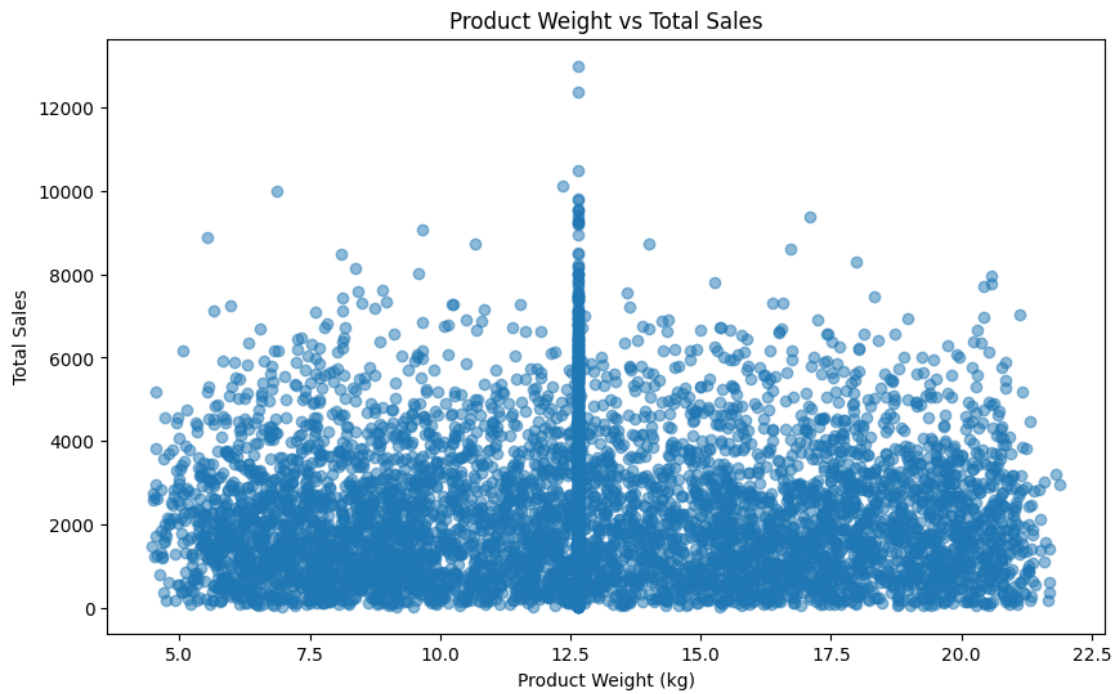


```
[23]: plt.figure(figsize=(10, 6))

plt.scatter(
    train['product_weight_kg'],
    train['total_sales'],
    alpha=0.5
)

plt.title('Product Weight vs Total Sales')
plt.xlabel('Product Weight (kg)')
plt.ylabel('Total Sales')

plt.show()
```

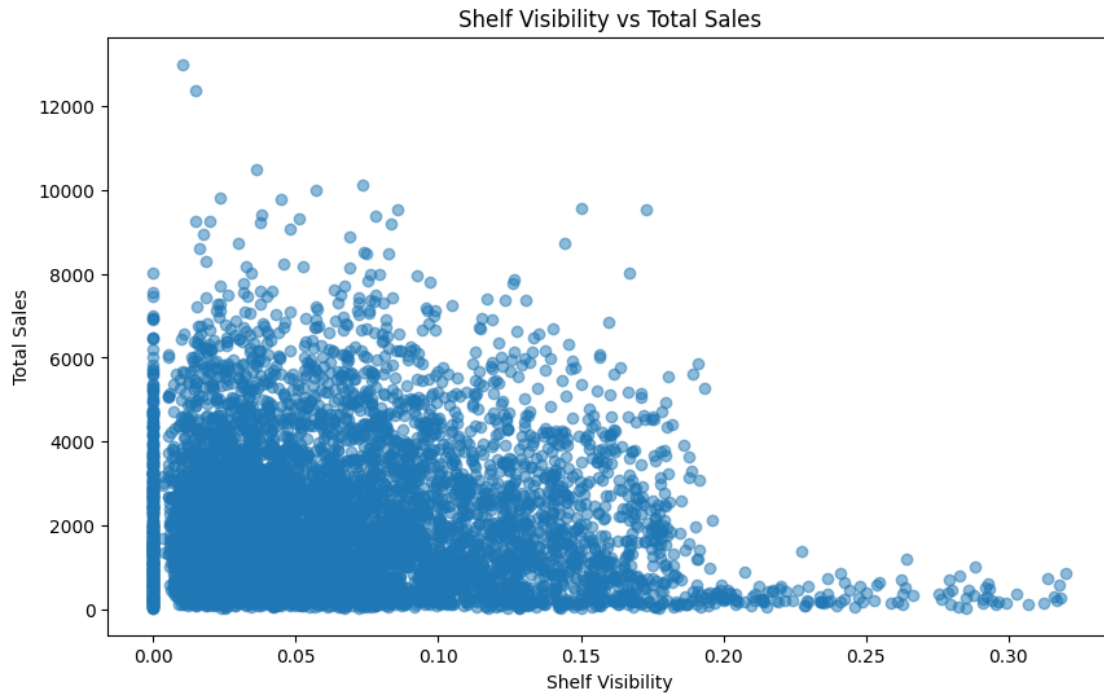


```
[24]: plt.figure(figsize=(10, 6))

plt.scatter(
    train['shelf_visibility'],
    train['total_sales'],
    alpha=0.5
)

plt.title('Shelf Visibility vs Total Sales')
plt.xlabel('Shelf Visibility')
plt.ylabel('Total Sales')

plt.show()
```



```
[25]: plt.figure(figsize=(10, 6))

plt.scatter(
    train['product_price'],
    train['total_sales'],
    alpha=0.5
)

plt.title('Product Price vs Total Sales')
plt.xlabel('Product Price')
plt.ylabel('Total Sales')

plt.show()
```

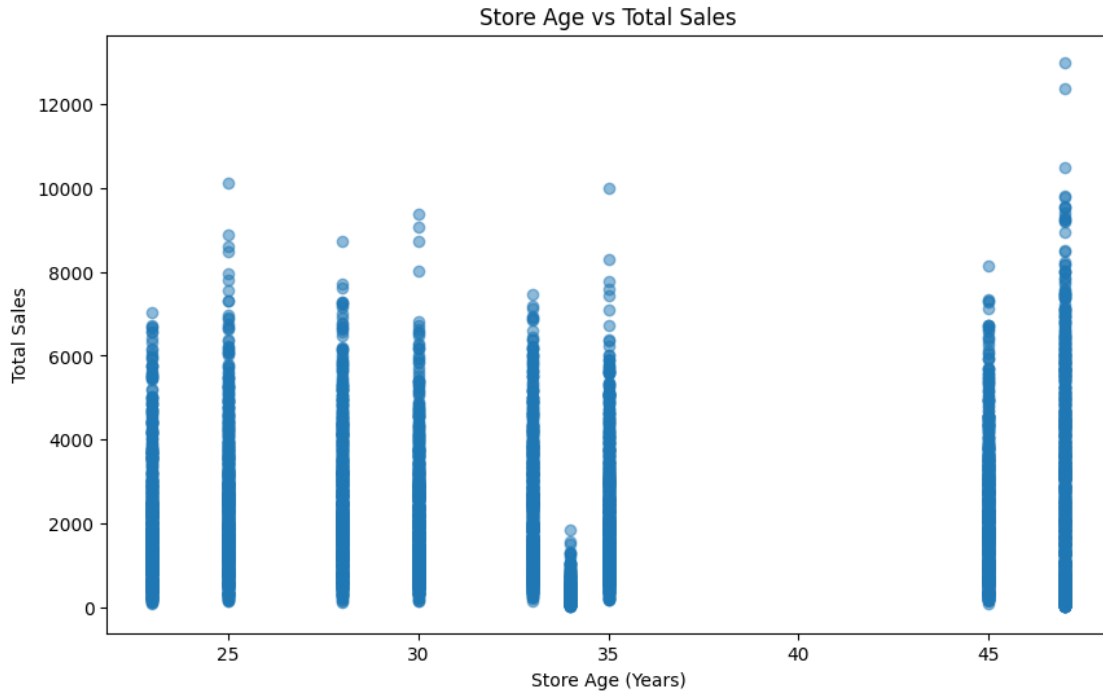


```
[26]: plt.figure(figsize=(10, 6))

plt.scatter(
    train['store_age_years'],
    train['total_sales'],
    alpha=0.5
)

plt.title('Store Age vs Total Sales')
plt.xlabel('Store Age (Years)')
plt.ylabel('Total Sales')

plt.show()
```



```
[27]: numerical_cols = [
        'product_weight_kg',
        'shelf_visibility',
        'product_price',
        'store_age_years',
        'total_sales'
    ]

    train[numerical_cols].corr()
```

```
[27]:
```

	product_weight_kg	shelf_visibility	product_price	\
product_weight_kg	1.000000	-0.013638	0.031926	
shelf_visibility	-0.013638	1.000000	0.003905	
product_price	0.031926	0.003905	1.000000	
store_age_years	-0.006839	0.071078	-0.012747	
total_sales	0.011306	-0.118124	0.569833	

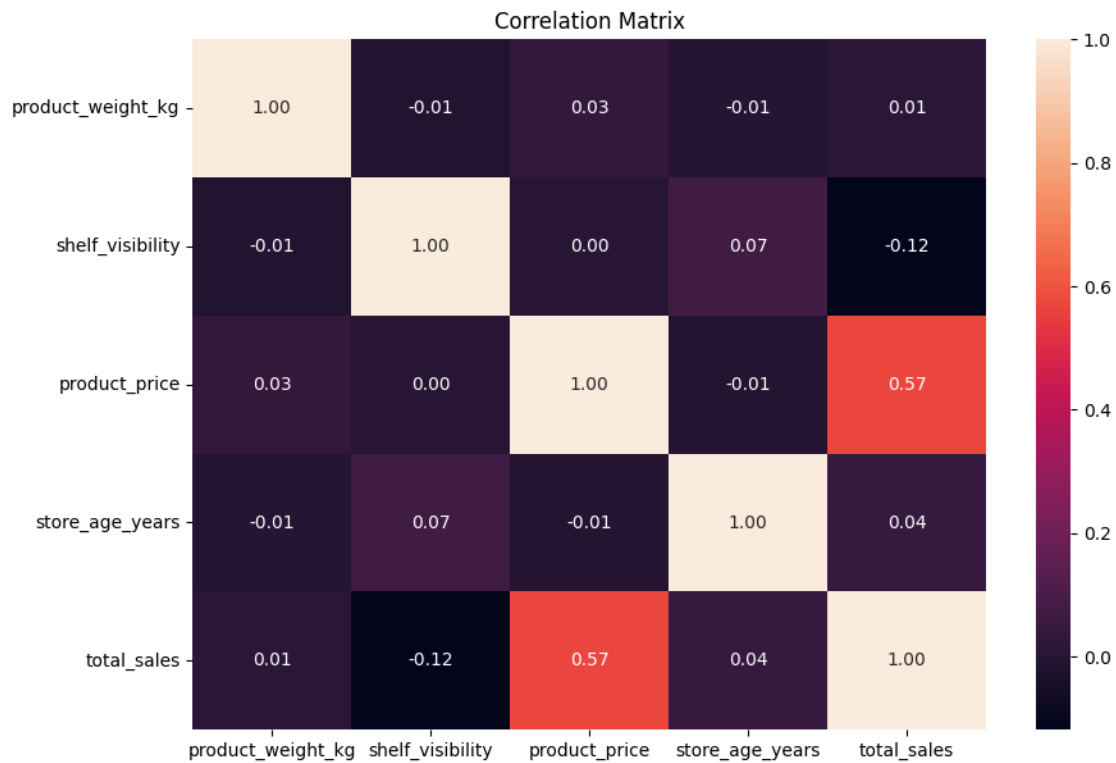
	store_age_years	total_sales
product_weight_kg	-0.006839	0.011306
shelf_visibility	0.071078	-0.118124
product_price	-0.012747	0.569833
store_age_years	1.000000	0.039443
total_sales	0.039443	1.000000

```
[28]: plt.figure(figsize=(10, 7))

sns.heatmap(
    train[numerical_cols].corr(),
    annot=True,
    fmt='.2f'
)

plt.title('Correlation Matrix')

plt.show()
```



```
[29]: train['product_code'].nunique()
```

```
[29]: 1555
```

```
[30]: train['store_code'].nunique()
```

```
[30]: 10
```

```
[31]: product_store_sales = (
    train.groupby(['product_code', 'store_code'])['total_sales']
    .mean()
```

```

        .reset_index()
    )

product_store_sales.head()

```

```

[31]: product_code store_code total_sales
0    PRD-00CVW9  STORE-AGY      1383.54
1    PRD-00CVW9  STORE-DKU      1122.13
2    PRD-00CVW9  STORE-HL7       834.54
3    PRD-00CVW9  STORE-YLW      1932.29
4    PRD-00EIUE  STORE-9RG      2782.79

```

```

[32]: train.groupby(
        ['Proper Product_Category', 'store_format']
    )['total_sales'].mean()

```

```

[32]: Proper Product_Category store_format
Baking Goods                Corner Shop      276.661912
                                Flagship Hypermarket 3277.200172
                                Standard Supermarket 2081.362840
                                Superstore      1738.689231
Breads                      Corner Shop      338.826296
...
Soft Drinks                 Superstore      1844.150250
Starchy Foods              Corner Shop      272.903000
                                Flagship Hypermarket 3597.843333
                                Standard Supermarket 2551.438831
                                Superstore      2122.018571
Name: total_sales, Length: 64, dtype: float64

```

```

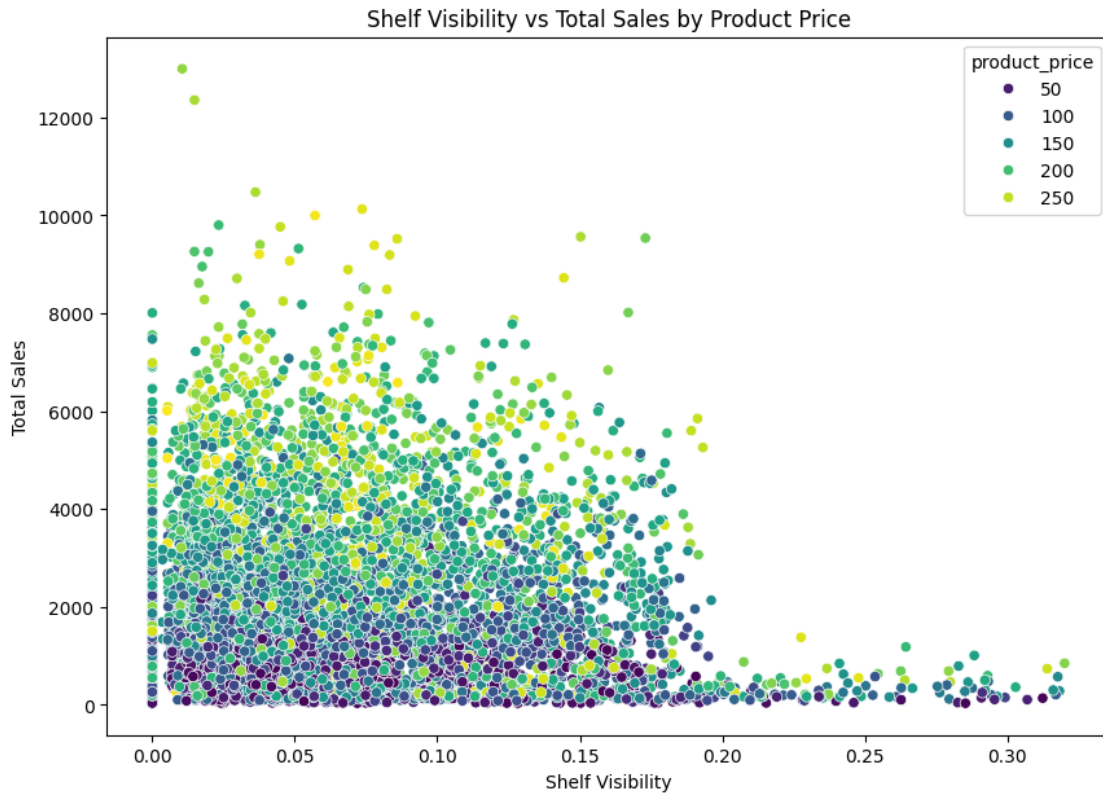
[33]: plt.figure(figsize=(10, 7))

sns.scatterplot(
    data=train,
    x='shelf_visibility',
    y='total_sales',
    hue='product_price',
    palette='viridis'
)

plt.title('Shelf Visibility vs Total Sales by Product Price')
plt.xlabel('Shelf Visibility')
plt.ylabel('Total Sales')

plt.show()

```

```
[34]: format_location_sales = train.pivot_table(
    index='store_format',
    columns='store_location_tier',
    values='total_sales',
    aggfunc='mean'
)

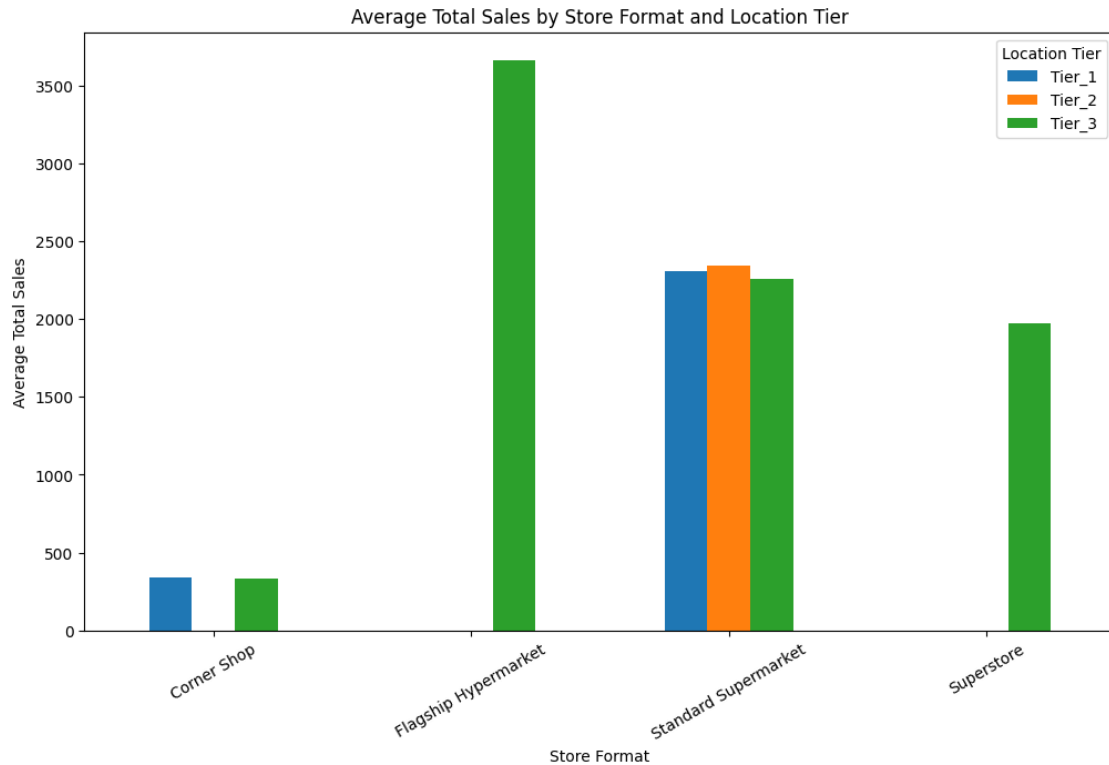
format_location_sales.plot(
    kind='bar',
    figsize=(12, 7)
)

plt.title('Average Total Sales by Store Format and Location Tier')
plt.xlabel('Store Format')
plt.ylabel('Average Total Sales')

plt.xticks(rotation=30)

plt.legend(title='Location Tier')

plt.show()
```



```
[35]: X = train.drop(columns=['total_sales', "id", "product_code", "store_code"])
      y = train['total_sales']
```

```
[36]: numerical_features = [
        'product_weight_kg',
        'shelf_visibility',
        'product_price',
        'store_age_years'
    ]

    categorical_features = [
        'fat_content',
        'Proper Product_Category',
        'store_size',
        'store_location_tier',
        'store_format'
    ]
```

```
[37]: preprocessor = ColumnTransformer(
        transformers=[
            (
                'num',
```

```

        StandardScaler(),
        numerical_features
    ),
    (
        'cat',
        OneHotEncoder(handle_unknown='ignore'),
        categorical_features
    )
]
)

```

```

[38]: linear_model = Pipeline([
        ('preprocessor', preprocessor),
        ('model', LinearRegression())
    ])

```

```

[39]: cv_results = cross_validate(
        linear_model,
        X,
        y,
        cv=5,
        scoring={
            'MAE': 'neg_mean_absolute_error',
            'RMSE': 'neg_root_mean_squared_error',
            'R2': 'r2'
        },
        return_train_score=False
    )

```

```

[40]: mae_scores = -cv_results['test_MAE']
        rmse_scores = -cv_results['test_RMSE']
        r2_scores = cv_results['test_R2']

```

```

[41]: print("MAE:", mae_scores)
        print("Mean MAE:", mae_scores.mean())

        print("RMSE:", rmse_scores)
        print("Mean RMSE:", rmse_scores.mean())

        print("R²:", r2_scores)
        print("Mean R²:", r2_scores.mean())

```

```

MAE: [795.31163943 823.9290899 853.60304102 861.31322846 864.82365667]
Mean MAE: 839.7961310985309
RMSE: [1077.94351002 1103.71320263 1135.13512116 1176.89367167 1154.94989182]
Mean RMSE: 1129.7270794574704
R²: [0.57688484 0.56446142 0.56280208 0.53628671 0.54230651]
Mean R²: 0.5565483128307074

```

```
[42]: results = []

results.append({
    'Model': 'Linear Regression',
    'MAE': mae_scores.mean(),
    'RMSE': rmse_scores.mean(),
    'R2': r2_scores.mean()
})

results
```

```
[42]: [{'Model': 'Linear Regression',
      'MAE': np.float64(839.7961310985309),
      'RMSE': np.float64(1129.7270794574704),
      'R2': np.float64(0.5565483128307074)}]
```

```
[43]: rf_preprocessor = ColumnTransformer(
    transformers=[
        ('num', 'passthrough', numerical_features),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)
    ]
)
```

```
[44]: random_forest_model = Pipeline([
    ('preprocessor', rf_preprocessor),
    ('model', RandomForestRegressor(
        n_estimators=300,
        random_state=42,
        n_jobs=-1
    ))
])
```

```
[45]: rf_results = cross_validate(
    random_forest_model,
    X,
    y,
    cv=5,
    scoring={
        'MAE': 'neg_mean_absolute_error',
        'RMSE': 'neg_root_mean_squared_error',
        'R2': 'r2'
    },
    return_train_score=False
)
```

```
[46]: rf_mae = -rf_results['test_MAE']
rf_rmse = -rf_results['test_RMSE']
```

```
rf_r2 = rf_results['test_R2']
```

```
[47]: print("Random Forest")
      print("MAE:", rf_mae)
      print("RMSE:", rf_rmse)
      print("R²:", rf_r2)
```

Random Forest

MAE: [742.32740232 761.56617681 806.40029338 799.34937417 797.00844023]

RMSE: [1070.54757205 1099.5960966 1142.69304499 1164.93053372 1121.89133822]

R²: [0.58267104 0.56770468 0.55696082 0.54566609 0.56813298]

```
[48]: print("\nMean MAE:", rf_mae.mean())
      print("Mean RMSE:", rf_rmse.mean())
      print("Mean R²:", rf_r2.mean())
```

Mean MAE: 781.3303373823662

Mean RMSE: 1119.9317171168714

Mean R²: 0.5642271243053429

```
[49]: results.append({
      'Model': 'Random Forest',
      'MAE': rf_mae.mean(),
      'RMSE': rf_rmse.mean(),
      'R2': rf_r2.mean()
    })
```

```
[50]: results_df = pd.DataFrame(results)
```

```
results_df
```

```
[50]:
```

	Model	MAE	RMSE	R2
0	Linear Regression	839.796131	1129.727079	0.556548
1	Random Forest	781.330337	1119.931717	0.564227

```
[51]: gradient_boosting_model = Pipeline([
      ('preprocessor', rf_preprocessor),
      ('model', GradientBoostingRegressor(
          n_estimators=300,
          learning_rate=0.05,
          max_depth=3,
          random_state=42
      ))
    ])
])
```

```
[52]: gb_results = cross_validate(
      gradient_boosting_model,
```

```

X,
y,
cv=5,
scoring={
    'MAE': 'neg_mean_absolute_error',
    'RMSE': 'neg_root_mean_squared_error',
    'R2': 'r2'
},
return_train_score=False
)

```

```

[53]: gb_mae = -gb_results['test_MAE']
      gb_rmse = -gb_results['test_RMSE']
      gb_r2 = gb_results['test_R2']

```

```

[54]: print("Gradient Boosting")
      print("MAE:", gb_mae)
      print("RMSE:", gb_rmse)
      print("R²:", gb_r2)

      print("\nMean MAE:", gb_mae.mean())
      print("Mean RMSE:", gb_rmse.mean())
      print("Mean R²:", gb_r2.mean())

```

Gradient Boosting

MAE: [718.61568919 749.69047821 785.83151484 783.05501844 789.94093425]

RMSE: [1039.54334778 1059.18302279 1099.09251606 1138.28145186 1113.06297886]

R²: [0.60649361 0.59889675 0.59012497 0.56621512 0.57490312]

Mean MAE: 765.426726986185

Mean RMSE: 1089.8326634724988

Mean R²: 0.5873267134690204

```

[55]: results.append({
      'Model': 'Gradient Boosting',
      'MAE': gb_mae.mean(),
      'RMSE': gb_rmse.mean(),
      'R2': gb_r2.mean()
    })

```

```

[56]: results_df = pd.DataFrame(results)

      results_df

```

```

[56]:
      Model      MAE      RMSE      R2
0  Linear Regression  839.796131  1129.727079  0.556548
1    Random Forest  781.330337  1119.931717  0.564227
2  Gradient Boosting  765.426727  1089.832663  0.587327

```

```
[57]: xgb_model = Pipeline([
      ('preprocessor', rf_preprocessor),
      ('model', XGBRegressor(
          n_estimators=300,
          learning_rate=0.05,
          max_depth=3,
          random_state=42,
          objective='reg:squarederror',
          n_jobs=-1
      ))
  ])

```

```
[58]: xgb_results = cross_validate(
      xgb_model,
      X,
      y,
      cv=5,
      scoring={
          'MAE': 'neg_mean_absolute_error',
          'RMSE': 'neg_root_mean_squared_error',
          'R2': 'r2'
      },
      return_train_score=False
  )

```

```
[59]: xgb_mae = -xgb_results['test_MAE']
      xgb_rmse = -xgb_results['test_RMSE']
      xgb_r2 = xgb_results['test_R2']

```

```
[60]: print("XGBoost")
      print("MAE:", xgb_mae)
      print("RMSE:", xgb_rmse)
      print("R²:", xgb_r2)

      print("\nMean MAE:", xgb_mae.mean())
      print("Mean RMSE:", xgb_rmse.mean())
      print("Mean R²:", xgb_r2.mean())

```

XGBoost

MAE: [720.03274134 749.65563247 780.81070828 777.79550095 785.29162867]
 RMSE: [1037.07162108 1066.24740972 1097.04772353 1136.40811377 1105.21189929]
 R²: [0.60836267 0.59352847 0.59164864 0.56764176 0.58087888]

Mean MAE: 762.7172423411721
 Mean RMSE: 1088.3973534802115
 Mean R²: 0.5884120827638771

```
[61]: results.append({
        'Model': 'XGBoost',
        'MAE': xgb_mae.mean(),
        'RMSE': xgb_rmse.mean(),
        'R2': xgb_r2.mean()
    })
```

```
[62]: results_df = pd.DataFrame(results)

results_df
```

```
[62]:
```

	Model	MAE	RMSE	R2
0	Linear Regression	839.796131	1129.727079	0.556548
1	Random Forest	781.330337	1119.931717	0.564227
2	Gradient Boosting	765.426727	1089.832663	0.587327
3	XGBoost	762.717242	1088.397353	0.588412

```
[63]: param_grid = {
        'model__n_estimators': randint(200, 800),
        'model__learning_rate': uniform(0.01, 0.09),
        'model__max_depth': randint(2, 7),
        'model__min_child_weight': randint(1, 8),
        'model__subsample': uniform(0.7, 0.3),
        'model__colsample_bytree': uniform(0.7, 0.3),
        'model__reg_alpha': uniform(0, 1),
        'model__reg_lambda': uniform(0.5, 2)
    }
```

```
[64]: xgb_tuning = RandomizedSearchCV(
        estimator=xgb_model,
        param_distributions=param_grid,
        n_iter=30,
        scoring='neg_mean_absolute_error',
        cv=5,
        random_state=42,
        n_jobs=-1,
        verbose=1
    )
```

```
[65]: xgb_tuning.fit(X, y)
```

Fitting 5 folds for each of 30 candidates, totalling 150 fits

```
[65]: RandomizedSearchCV(cv=5,
                        estimator=Pipeline(steps=[('preprocessor',
ColumnTransformer(transformers=[('num',
'passthrough',
['product_weight_kg',
```



```

'shelf_visibility',
'product_price',
'store_age_years']],
('cat',
OneHotEncoder(handle_unknown='ignore'),
['fat_content',
'Proper '
'Product_Category',
'store_size',
'store_location_tier',
'store_format']]])),

('model',
XGB...
'model__reg_alpha':
<scipy.stats._distn_infrastructure.rv_continuous_frozen object at
0x0000024515177490>,
'model__reg_lambda':
<scipy.stats._distn_infrastructure.rv_continuous_frozen object at
0x000002451559B100>,
'model__subsample':
<scipy.stats._distn_infrastructure.rv_continuous_frozen object at
0x0000024518814410>},
random_state=42, scoring='neg_mean_absolute_error',
verbose=1)

```

```

[66]: print("Best Parameters:")
      print(xgb_tuning.best_params_)

```

```

Best Parameters:
{'model__colsample_bytree': np.float64(0.877809017163818),
'model__learning_rate': np.float64(0.017276799369944373), 'model__max_depth': 4,
'model__min_child_weight': 1, 'model__n_estimators': 493, 'model__reg_alpha':
np.float64(0.3677831327192532), 'model__reg_lambda':
np.float64(1.764611661187159), 'model__subsample':
np.float64(0.8900589132282684)}

```

```

[67]: scoring='neg_mean_absolute_error'

```

```

[69]: best_cv_mae = -xgb_tuning.best_score_

      print("Best CV MAE:", best_cv_mae)

```

```

Best CV MAE: 758.1741103540453

```

```

[70]: best_xgb_model = xgb_tuning.best_estimator_

```

```

[71]: tuned_xgb_results = cross_validate(
      best_xgb_model,

```

```

X,
y,
cv=5,
scoring={
    'MAE': 'neg_mean_absolute_error',
    'RMSE': 'neg_root_mean_squared_error',
    'R2': 'r2'
},
return_train_score=False
)

tuned_xgb_mae = -tuned_xgb_results['test_MAE']
tuned_xgb_rmse = -tuned_xgb_results['test_RMSE']
tuned_xgb_r2 = tuned_xgb_results['test_R2']

print("Tuned XGBoost")
print("-----")

print("MAE:", tuned_xgb_mae)
print("Mean MAE:", tuned_xgb_mae.mean())

print("\nRMSE:", tuned_xgb_rmse)
print("Mean RMSE:", tuned_xgb_rmse.mean())

print("\nR2:", tuned_xgb_r2)
print("Mean R2:", tuned_xgb_r2.mean())

```

Tuned XGBoost

MAE: [719.74088142 743.69927584 776.31258309 773.60669823 780.98564298]

Mean MAE: 758.8690163122772

RMSE: [1036.1512486 1058.50354716 1092.39402812 1132.13493211 1100.69970953]

Mean RMSE: 1083.9766931057118

R²: [0.6090575 0.59941121 0.59510576 0.5708872 0.58429414]

Mean R²: 0.5917511602919227

```
[72]: results = []
```

```
[73]: results.append({
    'Model': 'Tuned XGBoost',
    'MAE': tuned_xgb_mae.mean(),
    'RMSE': tuned_xgb_rmse.mean(),
    'R2': tuned_xgb_r2.mean()
})
```

```
[74]: results_df = pd.DataFrame(results)

print(results_df)
```

	Model	MAE	RMSE	R2
0	Tuned XGBoost	758.869016	1083.976693	0.591751

```
[75]: results_df.sort_values(by='MAE')
```

	Model	MAE	RMSE	R2
0	Tuned XGBoost	758.869016	1083.976693	0.591751

```
[76]: xgb_final = best_xgb_model.named_steps['model']
```

```
[77]: feature_names = best_xgb_model.named_steps['preprocessor'].
      ↪get_feature_names_out()
```

```
[78]: feature_importance = pd.DataFrame({
      'Feature': feature_names,
      'Importance': xgb_final.feature_importances_
    })

feature_importance = feature_importance.sort_values(
    by='Importance',
    ascending=False
)

print(feature_importance)
```

	Feature	Importance
29	cat__store_format_Corner Shop	0.334927
31	cat__store_format_Standard Supermarket	0.142003
2	num__product_price	0.123453
3	num__store_age_years	0.083013
30	cat__store_format_Flagship Hypermarket	0.071662
23	cat__store_size_Medium	0.032456
7	cat__Proper Product_Category_Breads	0.014970
32	cat__store_format_Superstore	0.012319
26	cat__store_location_tier_Tier_1	0.011182
12	cat__Proper Product_Category_Fruits And Vegeta...	0.010729
22	cat__store_size_Large	0.010458
13	cat__Proper Product_Category_Hard Drinks	0.010402
28	cat__store_location_tier_Tier_3	0.009445
21	cat__Proper Product_Category_Starchy Foods	0.009329
19	cat__Proper Product_Category_Snack Foods	0.009329
20	cat__Proper Product_Category_Soft Drinks	0.009111
24	cat__store_size_Small	0.009087
27	cat__store_location_tier_Tier_2	0.008387
1	num__shelf_visibility	0.008243

15	cat__Proper Product_Category_Household	0.007783
18	cat__Proper Product_Category_Seafood	0.007637
0	num__product_weight_kg	0.007396
10	cat__Proper Product_Category_Dairy	0.007157
25	cat__store_size_Unknown	0.007015
6	cat__Proper Product_Category_Baking Goods	0.006752
17	cat__Proper Product_Category_Others	0.005842
11	cat__Proper Product_Category_Frozen Foods	0.005022
8	cat__Proper Product_Category_Breakfast	0.004864
9	cat__Proper Product_Category_Canned	0.004752
4	cat__fat_content_Low Fat	0.004644
14	cat__Proper Product_Category_Health And Hygiene	0.004515
16	cat__Proper Product_Category_Meat	0.003551
5	cat__fat_content_Regular	0.002563

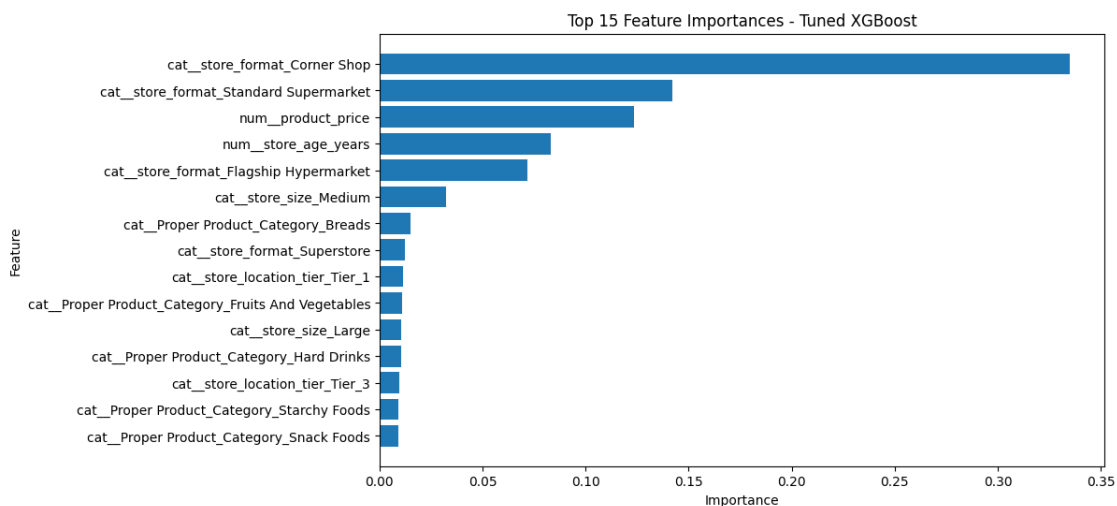
```
[79]: top_features = feature_importance.head(15)

plt.figure(figsize=(10, 6))

plt.barh(
    top_features['Feature'],
    top_features['Importance']
)

plt.xlabel('Importance')
plt.ylabel('Feature')
plt.title('Top 15 Feature Importances - Tuned XGBoost')
plt.gca().invert_yaxis()

plt.show()
```



```
[80]: best_xgb_model.fit(X, y)
```

```
[80]: Pipeline(steps=[('preprocessor',
                        ColumnTransformer(transformers=[('num', 'passthrough',
                                                         ['product_weight_kg',
                                                          'shelf_visibility',
                                                          'product_price',
                                                          'store_age_years']),
                                                         ('cat',
                                                          OneHotEncoder(handle_unknown='ignore'),
                                                          ['fat_content',
                                                           'Proper Product_Category',
                                                           'store_size',
                                                           'store_location_tier',
                                                           'store_format'])])),
                      ('model',
                       XGBRegressor(base_score=None, booster=None,
                                     gamma=None, grow_policy=None,
                                     importance_type=None,
                                     interaction_constraints=None,
                                     learning_rate=np.float64(0.017276799369944373),
                                     max_bin=None, max_cat_threshold=None,
                                     max_cat_to_onehot=None, max_delta_step=None,
                                     max_depth=4, max_leaves=None, min_child_weight=1,
                                     missing=nan, monotone_constraints=None,
                                     multi_strategy=None, n_estimators=493, n_jobs=-1,
                                     num_parallel_tree=None, ...)))]
```

```
[81]: X_test = test.drop(
        columns=['id', 'product_code', 'store_code']
    )
```

```
[89]: test_ids = test['id']
```

```
[95]: print(X.columns.tolist())
      print(X_test.columns.tolist())
```

```
['product_weight_kg', 'fat_content', 'shelf_visibility', 'Proper
Product_Category', 'product_price', 'store_age_years', 'store_size',
'store_location_tier', 'store_format']
['product_weight_kg', 'fat_content', 'shelf_visibility', 'product_price',
'Proper Product_Category', 'store_age_years', 'store_size',
'store_location_tier', 'store_format']
```

```
[96]: test_predictions = best_xgb_model.predict(X_test)
```

```
[97]: print(test_predictions[:10])
```

```
[2836.734  4394.6313  3242.3586  3180.0981  2440.4585  410.63553
 3692.6804  2924.6475   330.53662 2117.8784 ]
```

```
[98]: submission = pd.DataFrame({
      'id': test_ids,
      'total_sales': test_predictions
    })
```

```
[99]: print(submission.head())
      print(submission.shape)
```

```
      id  total_sales
0  row_00009  2836.733887
1  row_00015  4394.631348
2  row_00019  3242.358643
3  row_00020  3180.098145
4  row_00023  2440.458496
(1705, 2)
```

```
[ ]:
```